UNIVERSIDAD DE SAN BUENAVENTURA

Optativa II · Ingeniería de Software

MARKETPLACE

Documento de Arquitectura y Diseño

SDD — Descripción del diseño de software de Marketplace

Basado en IEEE 1016-2009 / ISO/IEC/IEEE 42010 · Código MKT-SAD-001 · Versión 2.0

Integrante	Rol Scrum
Sarah Isabella Peña	Product Owner
Juan Carlos Diaz	Scrum Master
Sebastian Grande Moreno	Developer

Docente: Ing. Jairo Armando Salcedo Aranda

Bogotá D.C., 3 de noviembre de 2026

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	02/11/2026	Equipo Marketplace	Versión inicial: autenticación, catálogo, carrito, pedidos y chatbot.
2.0	03/11/2026	Equipo Marketplace	Perfiles Comprador y Administrador, favoritos, historial de pedidos, panel y gestión de catálogo.

Tabla de contenido

Control de versiones....2
Tabla de contenido....2
1. Introducción....3
1.1 Propósito....3
1.2 Alcance....3
1.3 Partes interesadas y preocupaciones....3
2. Vista lógica — Arquitectura....3
2.1 Capas....4
3. Vista de desarrollo — Paquetes....4
4. Vista de datos....6
5. Vista de clases....7
6. Vista de procesos — Flujos clave....7
6.1 Inicio de sesión y asignación de rol....7
6.2 Compra....7
6.3 Gestión de producto (Administrador)....8
7. Decisiones de diseño....8
8. Seguridad....8

1. Introducción

1.1 Propósito

Describe la arquitectura, los componentes, el modelo de datos y las decisiones de diseño de Marketplace v2.0, incluyendo los nuevos perfiles Comprador y Administrador.

1.2 Alcance

Cubre la aplicación Android (módulo `app`) y su integración con Firebase. No cubre la consola de Firebase ni el pipeline de Azure, documentados en el Manual de instalación.

1.3 Partes interesadas y preocupaciones

Parte interesada	Preocupación
Product Owner	Que las funciones cubran a compradores y administradores.
Developer	Código mantenible, probado y fácil de extender.
Docente	Aplicación de estándares y buenas prácticas de ingeniería.
Usuario final	Rapidez, claridad y seguridad de su cuenta.

2. Vista lógica — Arquitectura

Se usa el patrón **MVVM** con UI declarativa: las pantallas Compose observan el estado de `MarketViewModel`, que orquesta Firebase, la persistencia local y la lógica de negocio. La lógica está en objetos Kotlin puros (sin Android), lo que permite probarla con JUnit sin emulador.

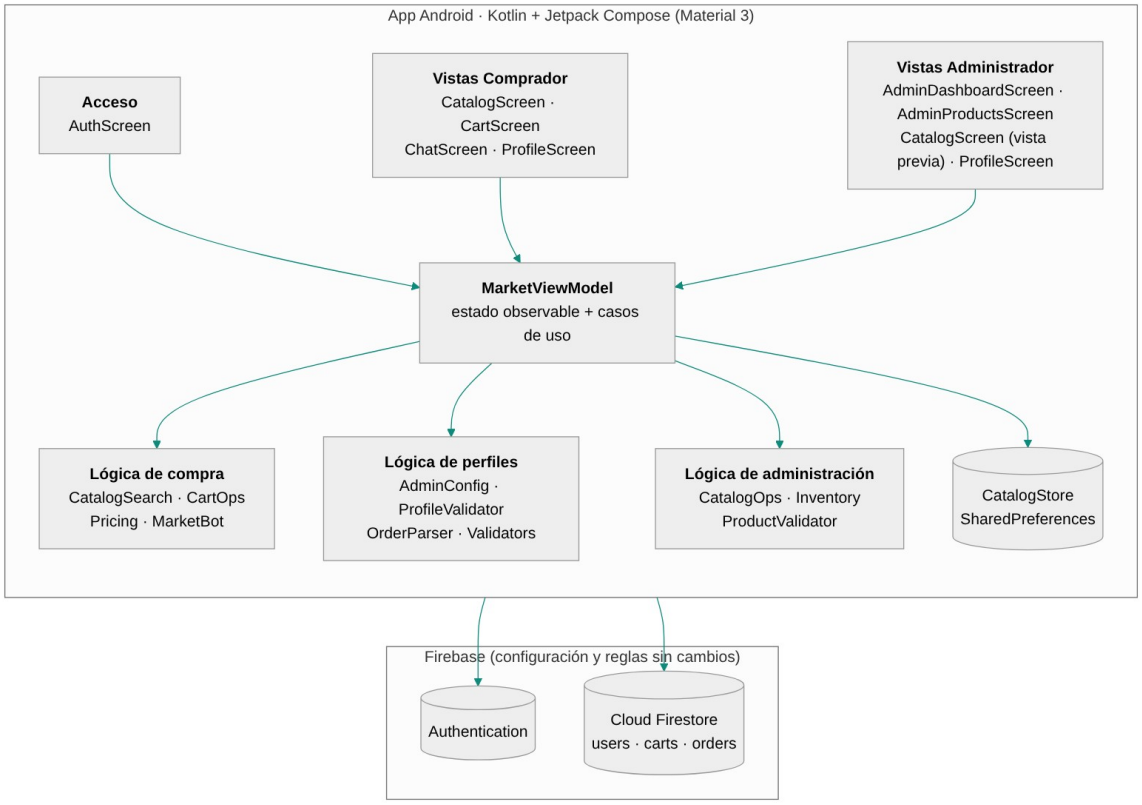


Figura 1. Arquitectura por capas

2.1 Capas

Capa	Responsabilidad	Elementos
Presentación	Dibujar el estado y capturar eventos del usuario.	AuthScreen, CatalogScreen, CartScreen, ChatScreen, ProfileScreen, AdminDashboardScreen, AdminProductsScreen, Theme
Estado / casos de uso	Mantener el estado observable, aplicar reglas y llamar a servicios.	MarketViewModel (AndroidViewModel)
Dominio (lógica pura)	Reglas de negocio deterministas y probadas.	CatalogSearch, CartOps, Pricing, Validators, AuthErrors, MarketBot, CatalogOps, Inventory, ProductValidator, ProfileValidator, OrderParser, AdminConfig
Datos	Modelos y persistencia.	Product, User, UserProfile, Role, ProductRepository, CatalogStore
Servicios externos	Identidad y base de datos en la nube.	Firebase Authentication, Cloud Firestore

3. Vista de desarrollo — Paquetes

```
app/src/main/java/com/optativa/marketplace/
├── MainActivity.kt          # navegación según el rol
├── data/
│   ├── Product.kt           # Product, User, UserProfile, ProductRepository
```

```
│   ├── Roles.kt              # Role, AdminConfig
│   └── CatalogStore.kt       # catálogo en SharedPreferences (JSON)
├── logic/
│   ├── Logic.kt              # búsqueda, carrito, precios, validadores, errores
│   ├── MarketBot.kt          # asistente Mari
│   ├── AdminLogic.kt         # ProductValidator, CatalogOps, Inventory
│   └── ProfileLogic.kt       # ProfileValidator, Initials, Dates, OrderParser
└── ui/
    ├── MarketViewModel.kt
    ├── AuthScreen.kt  CatalogScreen.kt  CartScreen.kt  ChatScreen.kt
    ├── ProfileScreen.kt      # perfil comprador / administrador
    ├── AdminScreens.kt       # panel y gestión de productos
    └── Theme.kt
```

4. Vista de datos

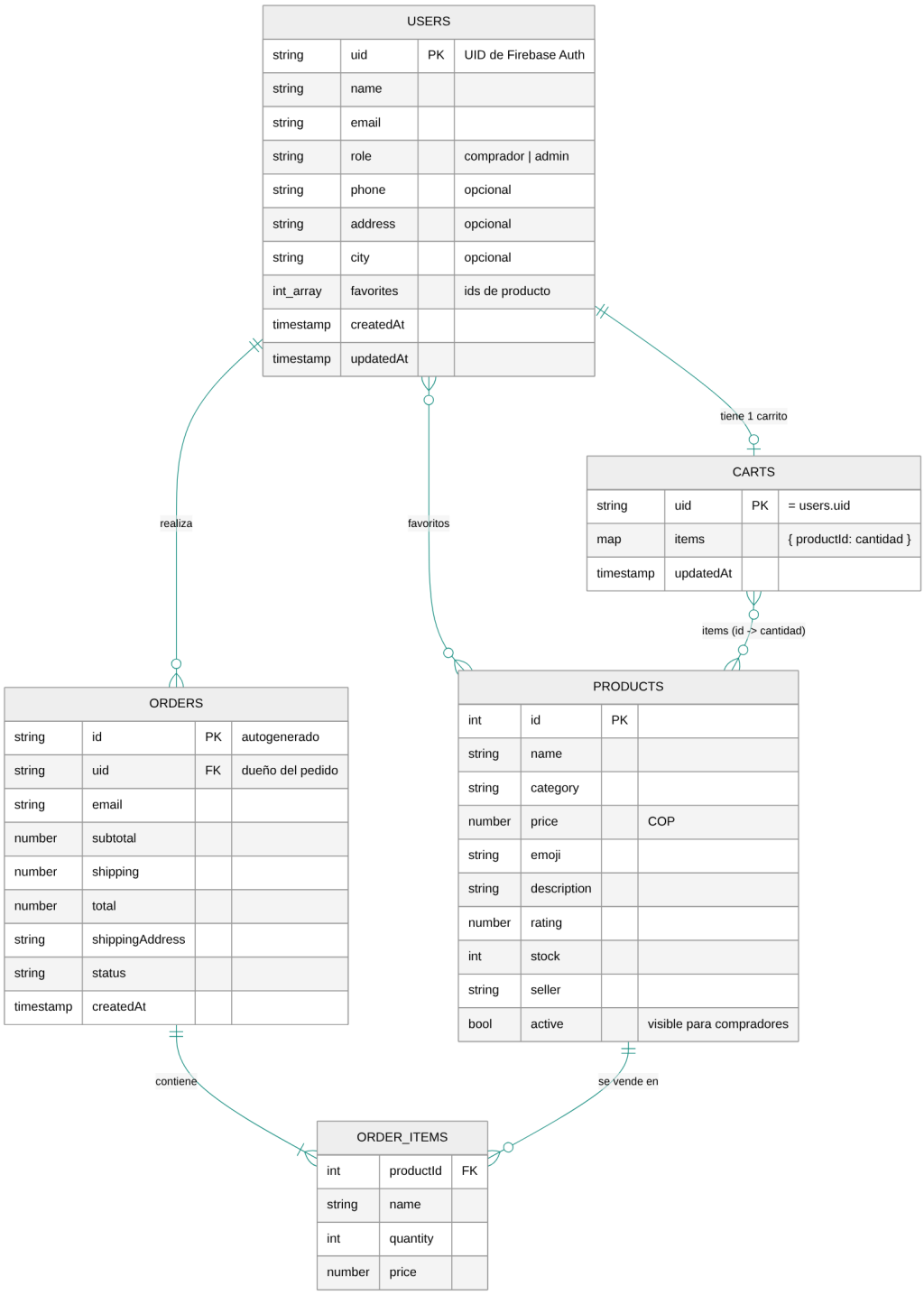


Figura 2. Modelo de datos (Firestore + catálogo local)

Colección / almacén	Clave	Contenido	Acceso (reglas)
users	uid	name, email, role, phone, address, city, favorites[], createdAt, updatedAt	Solo el dueño

Colección / almacén	Clave	Contenido	Acceso (reglas)
carts	uid	items {productId: cantidad}, updatedAt	Solo el dueño
orders	id automático	uid, email, items[], subtotal, shipping, total, shippingAddress, status, createdAt	Crear y leer propios; no editar ni borrar
CatalogStore (local)	products_v1	Lista JSON de productos con stock y estado activo	App en el dispositivo

5. Vista de clases

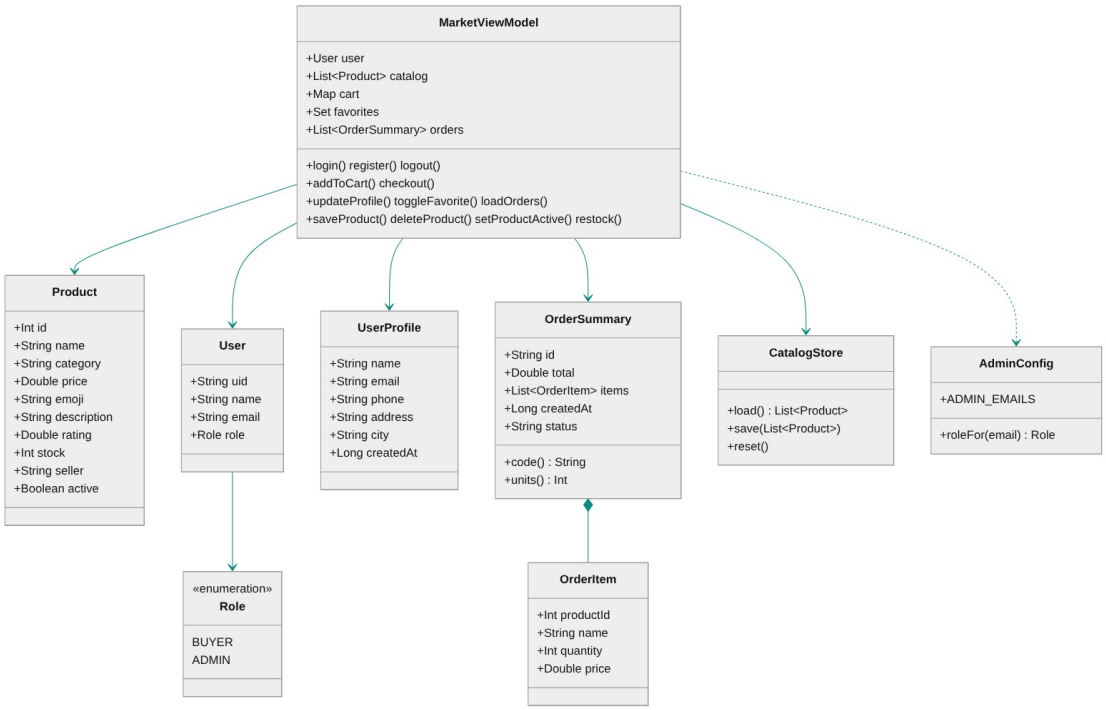


Figura 3. Diagrama de clases principal

6. Vista de procesos — Flujos clave

6.1 Inicio de sesión y asignación de rol

1. `AuthScreen` llama a `vm.login(email, password)`; se validan los datos con `Validators`.
2. Firebase Auth autentica; `toUser()` calcula `role = AdminConfig.roleFor(email)`.
3. `afterSignIn` carga `carts/{uid}` y `users/{uid}` (perfil y favoritos); si el documento no tiene el campo `role` se completa con `merge`.
4. `MarketApp` muestra la navegación del rol.

6.2 Compra

1. El comprador agrega productos (`CartOps.add` respeta stock) y el carrito se guarda en `carts/{uid}`.

2. `checkout()` arma el pedido con subtotal, envío, total y dirección del perfil y lo guarda en `orders/{id}`.
3. `CatalogOps.applyOrder` descuenta el stock y `CatalogStore` lo persiste; el carrito se vacía y se recarga el historial.

6.3 Gestión de producto (Administrador)

1. El formulario produce un `ProductDraft`; `ProductValidator.validate` devuelve el primer error o null.
2. `ProductValidator.toProduct` crea el `Product` (id con `CatalogOps.nextId`).
3. `vm.saveProduct` verifica `isAdmin`, aplica `CatalogOps.upsert` y guarda en `CatalogStore`.

7. Decisiones de diseño

ID	Decisión	Alternativas	Justificación
DD-01	Rol por lista de correos en código (AdminConfig).	Campo role en Firestore; Custom Claims.	Las reglas actuales permiten al usuario editar su users/{uid}; los Custom Claims requieren backend. Se evita modificar Firebase.
DD-02	Catálogo administrado en SharedPreferences.	Colección products en Firestore.	No requiere cambiar reglas de Firebase (restricción del proyecto). Se documenta como limitación R-01.
DD-03	Lógica de negocio en objetos Kotlin puros.	Lógica dentro del ViewModel.	Permite 46 pruebas JUnit rápidas sin emulador.
DD-04	Historial con whereEqualTo("uid") sin orderBy.	Consulta ordenada en servidor.	Evita crear índices compuestos; se ordena en el cliente con OrderParser.sort.
DD-05	Un solo ViewModel compartido.	Un ViewModel por pantalla.	App pequeña; el estado (usuario, carrito, catálogo) es compartido entre pestañas.
DD-06	Productos ocultos en vez de solo borrar.	Solo eliminar.	Permite retirar temporalmente un producto sin perder sus datos.

8. Seguridad

- Contraseñas gestionadas por Firebase Auth; nunca se almacenan en Firestore.
- Reglas de Firestore: cada usuario solo lee/escribe su perfil y carrito, y solo lee sus pedidos.
- Las funciones de administración verifican `isAdmin` en el ViewModel (`requireAdmin`) además de ocultarse en la interfaz.
- El campo `role` de Firestore es informativo; no concede permisos.